

Python — LOOPS (Part 1: Foundations)

Loops repeat a block of code. They are essential for lists, strings, dictionaries, sets, DSA and automation.

Python mainly uses two loops: *for* and *while*.

1. Why Loops?

- Avoid repeating the same code manually.
- Process every element of a collection.
- Repeat code while a condition is true.
- Build counters, searches, patterns and algorithms.

2. for Loop — Basic Syntax

```
for variable in iterable:
    # loop body
    print(variable)
```

3. Example

```
for i in range(5):
    print(i)    # 0 1 2 3 4
```

4. What Is an Iterable?

- An object that can be traversed one item at a time.
- Common iterables: list, tuple, string, set, dictionary and range.
- *for* takes one item from the iterable on each iteration.

5. Loop Through a List

```
numbers=[10,20,30]
for n in numbers:
    print(n)
```

6. Loop Through a String

```
word='Python'
for ch in word:
    print(ch)
```

Python — LOOPS (Part 2: range & for Patterns)

1. range() ☆

range() generates a sequence of numbers and is commonly used with for loops.

```
range(stop)
range(start, stop)
range(start, stop, step)
range(5)          # 0,1,2,3,4
range(2,6)        # 2,3,4,5
range(1,10,2)     # 1,3,5,7,9
```

2. Important range Rules

- *stop is excluded.*
- *Default start is 0.*
- *Default step is 1.*
- *Negative step counts backward.*

```
for i in range(5,0,-1):
    print(i)  # 5 4 3 2 1
```

3. Using Indexes

```
names=['A','B','C']
for i in range(len(names)):
    print(i,names[i])
```

4. enumerate() ☆

Use enumerate() when you need both the index and value.

```
for index,name in enumerate(names):
    print(index,name)
```

5. Nested for Loops

```
for i in range(3):
    for j in range(2):
        print(i,j)
```

6. Nested Data

```
matrix=[[1,2],[3,4]]
for row in matrix:
    for value in row:
        print(value)
```

Python — LOOPS (Part 3: while & Control Flow)

1. while Loop ☆

A while loop repeats as long as its condition is True.

```
count=1
while count<=5:
    print(count)
    count+=1
```

2. Infinite Loop Trap ?

- If the condition never becomes False, the loop may never stop.
- Update the state that controls the condition.

```
# Bad
x=1
while x<=5:
    print(x)
    # x never changes
```

3. while with Input

```
password=''
while password!='python':
    password=input('Password: ')
```

4. break ☆

break immediately terminates the nearest loop.

```
for n in range(10):
    if n==5:
        break
    print(n)
```

5. continue ☆

continue skips the rest of the current iteration and moves to the next one.

```
for n in range(6):
    if n==3:
        continue
    print(n)
```

6. pass

pass does nothing. It is a placeholder when Python requires a statement.

```
for n in range(3):
    pass
```

Python — LOOPS (Part 4: else & Collections)

1. Loop else ☆

else runs when the loop finishes normally, but not when it is terminated by break.

```
for n in range(3):  
    print(n)  
else:  
    print('Finished')
```

```
for n in range(5):  
    if n==2: break  
else:  
    print('Finished') # skipped
```

2. Conditions Inside Loops

```
for n in range(1,6):  
    if n%2==0:  
        print(n, 'even')  
    else:  
        print(n, 'odd')
```

3. Loop Through a Set

```
colors={'red', 'blue', 'green'}  
for color in colors:  
    print(color)
```

4. Loop Through a Tuple

```
data=(10,20,30)  
for x in data:  
    print(x)
```

5. Loop Through a Dictionary ☆

```
d={'name':'Jahanur', 'age':22}  
for key in d: print(key)  
for value in d.values(): print(value)  
for key,value in d.items(): print(key,value)
```

6. Dictionary Key vs Value

- *for key in d* iterates over keys.
- *d.values()* iterates over values.
- *d.items()* gives (key,value) pairs.

Python — LOOPS (Part 5: Useful Patterns)

1. Accumulator Pattern ☆

```
numbers=[10,20,30]
total=0
for n in numbers:
    total+=n
print(total)
```

2. Counter Pattern ☆

```
numbers=[1,2,3,4,5,6]
count=0
for n in numbers:
    if n%2==0: count+=1
print(count)
```

3. Find Maximum Without max()

```
numbers=[12,45,7,89,23]
largest=numbers[0]
for n in numbers:
    if n>largest: largest=n
print(largest)
```

4. Search Pattern

```
numbers=[10,20,30,40]
target=30
found=False
for n in numbers:
    if n==target:
        found=True
        break
```

5. Frequency Counting ☆

```
numbers=[1,2,2,3,3,3]
freq={}
for x in numbers:
    freq[x]=freq.get(x,0)+1
```

6. Build a New List

```
numbers=[1,2,3,4,5]
squares=[]
for n in numbers: squares.append(n*n)
```

7. Nested Loop — Pair Generation

```
nums=[1,2,3]
for a in nums:
    for b in nums:
        print(a,b)
```

Python — LOOPS (Part 6: Comprehensions & DSA)

1. List Comprehension ☆

A compact way to create a list using a loop, optionally with a condition.

```
squares=[x*x for x in range(1,6)]
evens=[x for x in range(10) if x%2==0]
```

2. Normal Loop vs Comprehension

```
result=[]
for x in range(5):
    result.append(x*x)

result=[x*x for x in range(5)]
```

3. Conditional Expression

```
result=['even' if x%2==0 else 'odd'
        for x in range(1,6)]
```

4. Set & Dictionary Comprehension

```
s={x*x for x in range(5)}
d={x:x*x for x in range(5)}
```

5. Loops + Complexity

- One loop over n items is commonly $O(n)$.
- Nested loops over n items are commonly $O(n^2)$.
- Two separate loops are commonly $O(n+n)=O(n)$.
- Dictionary lookup is typically $O(1)$ average-case.

6. Common DSA Loop Patterns

- Traversal: visit every element.
- Counting: maintain a counter.
- Accumulation: maintain a sum/result.
- Searching: compare and optionally break.
- Two pointers: move two indexes.
- Frequency map: loop + dictionary.

Python — LOOPS (Part 7: Mistakes & Practice)

1. Common Mistakes ?

- Forgetting the colon after for/while.
- Wrong indentation inside the loop.
- Creating an infinite while loop.
- Changing a collection carelessly while iterating.
- Using range(len(list)) when direct iteration is clearer.
- Confusing break, continue and pass.
- Forgetting range() excludes stop.

2. List Input + Loop Pattern

```
numbers=list(map(int,input().split()))
for n in numbers:
    print(n)
```

3. Beginner Practice

- 1. Print 1 to 100.
- 2. Print even numbers 1 to 50.
- 3. Find list sum.
- 4. Count positive and negative numbers.
- 5. Find largest without max().
- 6. Reverse a list without reverse() or slicing.

4. Intermediate Practice

- 7. Remove duplicates without set().
- 8. Find second largest unique number.
- 9. Count frequency of each element.
- 10. Find first repeated element.
- 11. Find common elements of two lists.
- 12. Print multiplication tables using nested loops.

5. DSA Practice

- 13. Two Sum using a dictionary.
- 14. First non-repeating character.
- 15. Move all zeros to the end.
- 16. Find duplicate values.
- 17. Maximum subarray sum.
- 18. Pair-sum using nested loops.

6. Quick Revision ☆

- for → iterate over an iterable.
- while → repeat while condition is True.
- range() → number sequence; stop excluded.
- break → exit loop.
- continue → skip current iteration.
- pass → placeholder.
- else → runs after normal completion.
- enumerate() → index + value.
- Comprehension → compact loop-based construction.